

Kafka Streams: Topology, Building in Java, Mapping, and Filtering

Main Takeaway:

A Kafka Streams **topology** is a directed acyclic graph (DAG) defining streams flow through sources, processors, and sinks. You build and run a topology in Java via the `StreamsBuilder` DSL. For record-level transformations, prefer `mapValues` over `map` when only the value changes (it avoids repartitioning)^[1]. To drop unwanted records, use `filter` or `filterNot` to create stateless, concise pipelines.

1. Processor Topology

A **processor topology** is the core of every Kafka Streams application. It is an acyclic graph where:

- **Source processors** consume from Kafka topics (the graph's inputs).
- **Stream processors** apply operations (e.g., `map`, `filter`).
- **Sink processors** write results back to Kafka topics (the graph's outputs).

At runtime, each topology is instantiated into one or more tasks aligned with input partitions for parallelism and fault tolerance^[2].

Example logical topology description:

```
Sub-topology: 0
Source: KSTREAM-SOURCE-0000000000 (topics: [input-topic])
--> KSTREAM-PROCESSOR-0000000001
Sink: KSTREAM-SINK-0000000001 (topic: output-topic)
<-- KSTREAM-SOURCE-0000000000
```

(source, processor, sink nodes connected in a DAG)^[3]

2. Building a Kafka Streams Application in Java

Minimal Java Example

```
import org.apache.kafka.common.serialization.Serdes;
import org.apache.kafka.streams.*;
import org.apache.kafka.streams.kstream.*;

import java.util.Properties;

public class SimpleStreamApp {
    public static void main(String[] args) {
```

```

// 1. Configuration
Properties props = new Properties();
props.put(StreamsConfig.APPLICATION_ID_CONFIG, "simple-app");
props.put(StreamsConfig.BOOTSTRAP_SERVERS_CONFIG, "localhost:9092");
props.put(StreamsConfig.DEFAULT_KEY_SERDE_CLASS_CONFIG, Serdes.String().getClass());
props.put(StreamsConfig.DEFAULT_VALUE_SERDE_CLASS_CONFIG, Serdes.String().getClass());

// 2. Build topology
StreamsBuilder builder = new StreamsBuilder();
builder.stream("input-topic")
    .mapValues(value -> value.toUpperCase())
    .to("output-topic");

// 3. Start streams
KafkaStreams streams = new KafkaStreams(builder.build(), props);
streams.start();
}
}

```

This sets up a single-source, single-sink topology that uppercases each message value before writing out [\[4\]](#).

3. Mapping Records: `map` vs. `mapValues`

- `map`
 - Signature: `<KR,VR> KStream<KR,VR> map(KeyValueMapper<? super K,? super V,KeyValue<? extends KR,? extends VR>> mapper)`
 - Transforms both key and value; returns new key-value pairs.
 - May trigger repartitioning if keys change.
- `mapValues`
 - Signature: `<VR> KStream<K,VR> mapValues(ValueMapper<? super V,? extends VR> mapper)`
 - Transforms only the value, preserving the original key and partitioning.
 - **Prefer** `mapValues` for value-only changes to avoid repartitioning overhead [\[1\]](#).

Example:

```

KStream<String,String> stream = builder.stream("input-topic");
// Value-only transform; preserves keys/partitions
stream.mapValues(v -> "prefix-" + v)
    .to("mapped-topic");

```

4. Filtering Records

Use `stateless filter` or `filterNot` to include or exclude records based on a predicate:

- `filter((key,value) -> predicate)` retains records that satisfy the predicate.
- `filterNot((key,value) -> predicate)` retains records that do *not* satisfy it.

Example:

```
KStream<String,String> stream = builder.stream("input-topic");
// Keep only records whose value contains "SUCCESS"
stream.filter((k, v) -> v.contains("SUCCESS"))
    .to("success-topic");

// Alternatively, drop records starting with "DEBUG"
stream.filterNot((k, v) -> v.startsWith("DEBUG"))
    .to("no-debug-topic");
```

This creates new streams with only the desired records, leaving the original unchanged^[5] ^[6].

By understanding topologies, using the Java DSL to build your application, choosing `mapValues` for efficient value transformations, and applying `filter` for conditional routing, you can craft succinct and high-performance Kafka Streams applications in just a few lines of code.

✱

1. <https://docs.confluent.io/platform/current/streams/developer-guide/dsl-api.html>
2. <https://docs.confluent.io/platform/current/streams/architecture.html>
3. <https://books.japila.pl/kafka-streams-internals/demo/developing-kafka-streams-application/>
4. <https://kafka.apache.org/21/javadoc/org/apache/kafka/streams/KafkaStreams.html>
5. <https://developer.confluent.io/confluent-tutorials/filtering/kstreams/>
6. <https://stackoverflow.com/questions/40918158/kafka-how-to-use-filter-and-filternot-at-the-same-time>